

Proposal: AI-assisted Parallelization of bzip2 Compression

Team 05

1. Project Title

AI-assisted Parallelization of bzip2 Compression with Profiling-guided Optimization

2. Background

`bzip2` is a widely used compression algorithm known for its relatively high compression ratio. Internally, bzip2 processes data in blocks, which makes it a potential candidate for parallelization. Since different data blocks can be compressed independently, multiple blocks may be assigned to different worker threads and processed in parallel.

However, parallelizing compression is not trivial. The system must handle output ordering, memory usage, load balancing, synchronization overhead, and possible I/O bottlenecks. In addition, block size may affect both compression ratio and parallel efficiency.

In this project, we focus not only on parallelizing bzip2, but also on investigating how AI can assist in the parallelization process. Large language models may help identify parallelizable components, generate multithreaded code, suggest pipeline designs, and optimize performance. However, AI may not fully understand the target machine, runtime behavior, synchronization costs, or the actual performance bottlenecks after parallelization. Therefore, we plan to guide AI using profiling results and system-level feedback.

3. Project Objective

The objective of this project is to explore different ways of using AI to assist in parallelizing a bzip2-like compression system, and to evaluate whether AI-generated or AI-assisted parallel versions can improve performance while preserving correctness.

The project will compare:

1. A sequential bzip2 baseline.
2. An existing parallel implementation, such as `lbzip2`.
3. A naive AI-generated parallel version.
4. A constraint-guided AI parallel version.

5. A profiling-guided AI optimized version.

The system will:

- Split a large input file into fixed-size blocks.
 - Compress different blocks independently using multiple threads.
 - Preserve the original block order in the output file.
 - Verify correctness by decompressing the compressed file and comparing it with the original input.
 - Evaluate runtime, throughput, speedup, compression ratio, memory usage, and scalability under different thread counts and block sizes.
 - Analyze how AI performs under different levels of guidance, such as direct prompting, design constraints, and profiling feedback.
-

4. AI-assisted Parallelization Strategy

The main idea is to use AI as a coding and optimization assistant during the parallelization process. Instead of only asking AI to “make the code faster,” we will design a structured workflow to guide AI step by step.

Stage 1: Naive AI Parallelization

In the first stage, we will provide AI with the sequential bzip2 compression code and ask it to parallelize the program.

Example prompt:

Please parallelize this bzip2 compression program using multiple threads.
Preserve correctness and improve performance.

This stage tests whether AI can independently identify the parallelizable parts of the program.

Stage 2: Constraint-guided AI Parallelization

In the second stage, we will provide AI with clearer design requirements.

Example prompt:

Please parallelize the compressor using block-level parallelism.

Requirements:

1. Split the input file into fixed-size blocks.
2. Assign each block a unique block ID.

3. Use worker threads to compress blocks independently.
4. Use a writer thread to output compressed blocks in the original order.
5. Avoid changing the compression result.
6. Avoid race conditions and unnecessary global locks.

This stage tests whether explicit system design constraints can help AI generate more correct and maintainable parallel code.

Stage 3: Profiling-guided AI Optimization

In the third stage, we will run the AI-generated parallel version and collect profiling information, such as:

- CPU utilization
- runtime breakdown
- I/O waiting time
- lock contention
- memory usage
- thread idle time
- scalability under different thread counts

Then, we will provide these profiling results to AI and ask it to improve the implementation.

Example prompt:

The profiling result shows that the writer thread becomes the bottleneck and worker threads are often idle after 8 threads.

Please optimize the parallel design while preserving correctness.

This stage tests whether profiling feedback can help AI move from superficial code changes to bottleneck-driven optimization.

5. Evaluation Metrics

We will evaluate both system performance and AI effectiveness.

5.1 Performance Metrics

Metric	Description
Runtime	Total compression time

Throughput	Input size divided by compression time, measured in MB/s
Speedup	Sequential runtime divided by parallel runtime
Parallel efficiency	Speedup divided by number of threads
Compression ratio	Compressed file size divided by original file size
Memory usage	Peak memory consumption
Correctness	Whether decompressed output matches the original file

6. Expected Results

We expect that the parallel version will reduce compression time for large compressible files, especially when the workload is CPU-bound. However, the speedup may not be perfectly linear because of I/O bottlenecks, synchronization overhead, memory bandwidth limitation, output ordering cost, and uneven compression time across blocks.

We also expect that naive AI-generated code may not produce the best result immediately. AI may generate code that compiles but performs poorly, or code that improves performance but breaks correctness. By providing clearer design constraints and profiling results, we expect AI to generate more effective and reliable parallelization strategies.

We expect to observe the following trends:

- Naive AI prompting may produce incomplete or unsafe parallelization.
 - Constraint-guided prompting may improve correctness and code structure.
 - Profiling-guided prompting may improve actual runtime performance.
 - Block size and thread count will significantly affect speedup and compression ratio.
 - Existing tools such as [lzip2](#) or [pbzip2](#) may still outperform our implementation, but they can serve as useful reference points.
-

7. Expected Challenges and Solutions

Expected Challenge	Why It Is a Problem	Proposed Solution
AI may not understand the target machine	AI does not know the number of CPU cores, cache behavior, memory bandwidth, or disk speed	Provide machine information and run experiments under different thread counts

AI may not know the actual performance after parallelization	AI can suggest code changes, but it cannot know whether they improve runtime without measurement	Provide profiling results and benchmark data as feedback
AI may ignore I/O bottlenecks	After compression is parallelized, reading or writing may become the bottleneck	Use profiling tools to measure I/O waiting time and guide AI toward pipeline optimization

8. References

- bzip2 source: <https://gitlab.com/bzip2/bzip2>
- lbzip2 source: <https://github.com/kjn/lbzip2/tree/master>
- pbzip2 source: <https://github.com/ruanhuabin/pbzip2>